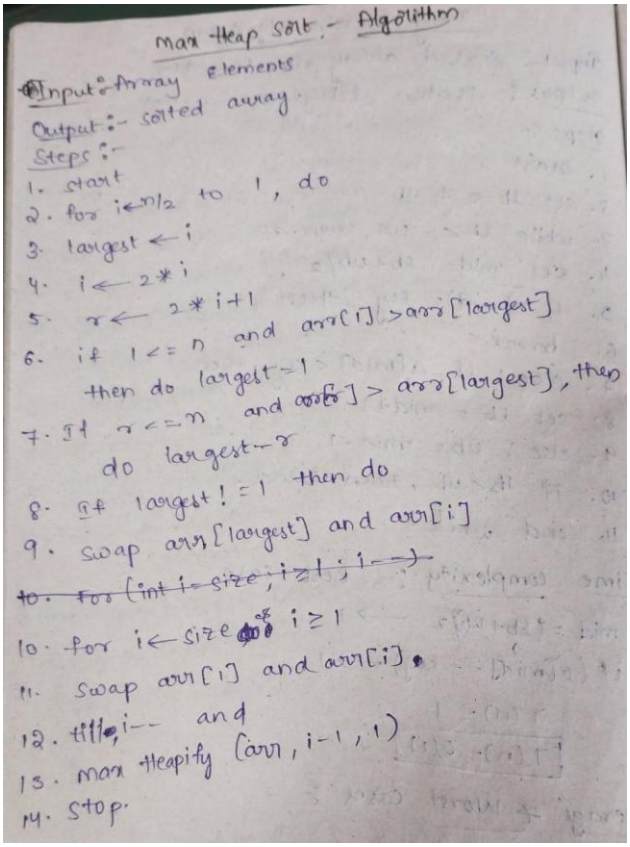
Practical - 3

Aim: Implementation of max-heap sort algorithm.

Algorithms:



Code:

```
#include <iostream> using namespace std;

void heapify(int arr[], int n, int i)
{    int largest = i;    int left = 2 * i + 1;    int
right = 2 * i + 2;

   if (left < n && arr[left] > arr[largest])
largest = left;
```

```
    if (right < n && arr[right] > arr[largest])
largest = right;

    if (largest != i)     {         swap(arr[i],
arr[largest]);         heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{     for (int i = n / 2 - 1; i >= 0; i--)
heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {         swap(arr[0], arr[i]);         heapify(arr, i,
0);
    }
}

int main() {     int n;

    cout << "Enter number of elements: ";     cin
>> n;

    int* arr = new int[n];

    cout << "Enter " << n << " elements: ";     for
(int i = 0; i < n; i++)         cin >> arr[i];

    heapSort(arr, n);

    cout << "Sorted array: ";     for (int i = 0; i < n;
i++)         cout << arr[i] << " ";

    cout << endl;
```

```
    delete[] arr;

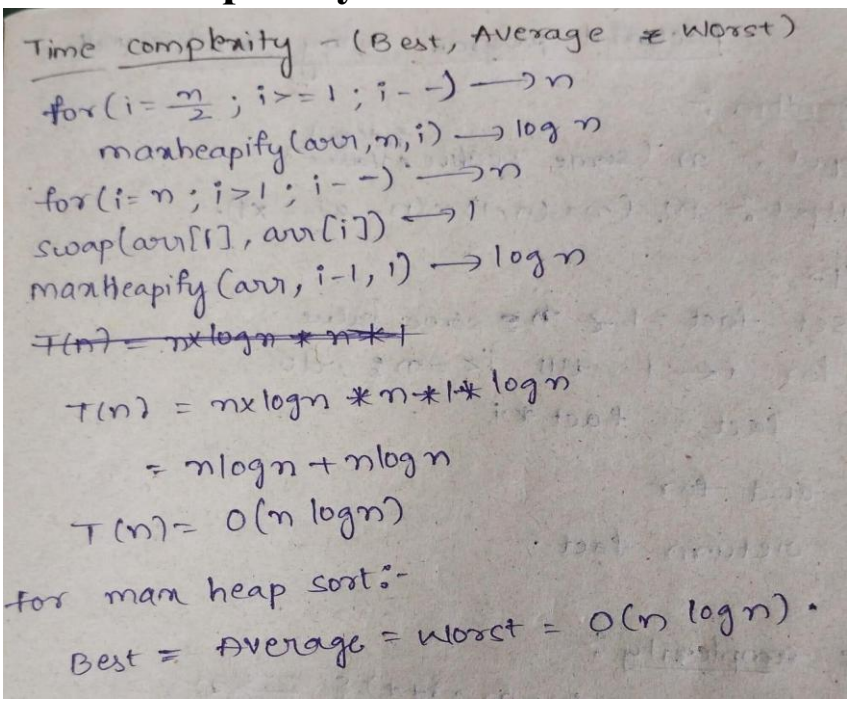
    return 0;
}
```

Output:

```
Enter number of elements: 4
Enter 4 elements: 1
4
45
3
Sorted array: 1 3 4 45


=== Code Execution Successful ===
```

Time Complexity:



CONCLUSION:

The Max-Heap Sort algorithm was successfully implemented and tested. It sorts the elements by creating a max-heap and repeatedly placing the largest element in its correct position. It is efficient for large amounts of data and has a time complexity of O(n log n).